AI DZ ACADEMY

LES RÉSEAUX DE NEURONES
RNN

Atelier Gratuit — 2 Heures :

De la mémoire artificielle à l'analyse de séquences

Architecture RNN, problème du gradient qui disparaît,
LSTM, GRU et implémentation complète avec PyTorch

Instructeur principal
Abdelhak Samir Douli

Niveau :	Intermédiaire
Prérequis :	MLP, boucle d'entraînement PyTorch, descente de gradient
Durée :	2 heures (théorie + pratique guidée)
Objectif :	Maîtriser RNN, LSTM et GRU avec PyTorch

Version 1.0 | 15 juin 2026

Table des matières

1 Pourquoi les Modèles de Séquences ? 3
1.1 Les données séquentielles sont partout . . . 3
1.2 L'ordre est fondamental . . . 3

2 Le Problème des Réseaux Classiques face aux Séquences 3
2.1 Les réseaux denses (MLP) et leur mémoire nulle . . . 3

3 Les Réseaux de Neurones Récurrents (RNN) 4
3.1 L'idée centrale : la mémoire artificielle . . . 4
3.2 La cellule RNN : forme repliée et dépliée . . . 5
3.3 L'équation mathématique du RNN . . . 5

4 L'Entraînement : Rétropropagation dans le Temps (BPTT) 6
4.1 Principe général . . . 6
4.2 Comment le gradient remonte le temps . . . 6

5 Les Différentes Architectures RNN 7
5.1 Vue d'ensemble . . . 7
5.2 Architecture 1-à-1 : la référence . . . 7
5.3 Architecture 1-à-N : génération à partir d'un stimulus . . . 7
5.4 Architecture N-à-1 : classifier une séquence . . . 8
5.5 Architecture N-à-N synchrone : annoter chaque élément . . . 8
5.6 Architecture N-à-N décalé (Seq2Seq) : transformer une séquence . . . 8

6 Modèles de Langage et Génération de Séquences 9
6.1 Qu'est-ce qu'un modèle de langage ? . . . 9
6.2 Niveau caractère ou niveau mot ? . . . 9
6.3 Comment un RNN devient un modèle de langage ? . . . 9

7 Sampling : Générer de Nouvelles Séquences 10
7.1 Du modèle appris à la génération . . . 10
7.2 La température : contrôler la créativité . . . 11

8 Le Problème du Gradient qui Disparaît 11
8.1 Rappel : comment les gradients circulent dans un RNN . . . 11
8.2 Pourquoi le gradient disparaît . . . 12
8.3 Conséquence concrète : mémoire courte . . . 12
8.4 L'autre problème : le gradient qui explose . . . 12
8.5 Résumé : quand le RNN vanille atteint ses limites . . . 13

9 GRU — Gated Recurrent Unit 13
9.1 Motivation : résoudre le problème avec des portes . . . 13
9.2 Les deux portes du GRU . . . 14
9.3 Interprétation intuitive . . . 14
9.4 Pourquoi le GRU résout le gradient qui disparaît . . . 14

10 LSTM — Long Short-Term Memory **15**
10.1 Motivation : une mémoire à deux niveaux 15
10.2 Les trois portes du LSTM 15
10.3 Pourquoi le LSTM résout le gradient qui disparaît 16
10.4 Comparaison finale : RNN, GRU, LSTM 16

11 Récapitulatif et Points Clés **16**

1 Pourquoi les Modèles de Séquences ?

1.1 Les données séquentielles sont partout

Dans la vie réelle, une grande partie des données que nous traitons sont **séquentielles** : elles s'inscrivent dans un ordre précis où chaque élément dépend de ce qui précède.

Type de données	Exemple de séquence	Application
Texte	Mots d'une phrase	Traduction, chatbot
Audio	Trames sonores	Reconnaissance vocale
Vidéo	Images consécutives	Analyse d'activité
Finance	Prix boursiers	Prévision de tendance
Médecine	Battements cardiaques	Détection d'anomalies
Génétique	Bases ADN (A, T, G, C)	Prédiction de protéines

1.2 L'ordre est fondamental

La caractéristique principale d'une séquence : **l'ordre change le sens**.

Exemple : L'ordre des mots change tout

Comparez ces deux phrases :

— « Le chat mange la souris. » ⇒ Le chat est le prédateur.

— « La souris mange le chat. » ⇒ La souris est le prédateur.

Les mêmes mots, dans un ordre différent, produisent un sens **complètement opposé**.

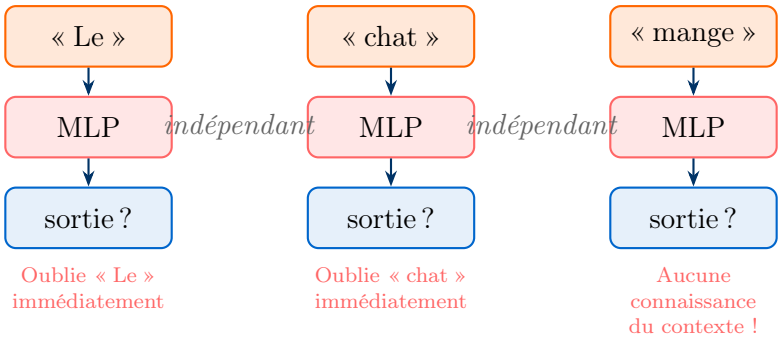
Analogie du monde réel : La lecture d'une histoire

Imaginez que vous lisez : « Marie a soif. Elle prend un verre d'eau et le... » Pour prédire le mot suivant (« boit », « renverse », etc.), vous avez besoin du **contexte des mots précédents**. Sans ce contexte, le mot « le » seul ne vous dit rien.
Un modèle de séquence doit faire exactement cela : mémoriser le passé pour interpréter le présent.

2 Le Problème des Réseaux Classiques face aux Séquences

2.1 Les réseaux denses (MLP) et leur mémoire nulle

Un réseau de neurones dense (MLP) reçoit une entrée de **taille fixe** et produit une sortie. Il n'a **aucune notion d'ordre ou de contexte temporel**. Voici ce qui se passe si on essaie de lui faire traiter une phrase mot par mot :



Attention : Les 3 limites fondamentales du MLP face aux séquences

1. **Taille d'entrée fixe :** un MLP est conçu pour une entrée de dimension précise. Comment gérer des phrases de 5 mots et d'autres de 50 mots avec le même réseau ?
2. **Aucune mémoire :** chaque entrée est traitée **indépendamment**. Le réseau ne sait pas ce qu'il a traité à l'étape précédente.
3. **Aucun sens de l'ordre :** donner les mots dans n'importe quel ordre produirait la même sortie — ce qui est absurde pour du texte.

Exemple : Analyse de sentiment sans mémoire

Prenons la phrase : « **Ce film n'est pas mauvais** ».
Sans contexte, si un MLP voit le mot « mauvais » isolément, il prédit **sentiment négatif**.
Mais en réalité, la phrase est positive grâce au « n'est pas » qui précède !
Un modèle séquentiel avec mémoire comprendrait le contexte complet : « n'est pas » + « mauvais » = **positif**.

3 Les Réseaux de Neurones Récurrents (RNN)

3.1 L'idée centrale : la mémoire artificielle

La solution est simple et élégante : donner au réseau une **mémoire**. À chaque pas de temps, le réseau reçoit non seulement l'entrée actuelle, mais aussi un **résumé de tout ce qu'il a vu avant** : c'est l'**état caché** $\mathbf{a}_t$.

Réseau de Neurones Récurrent (RNN)

Un **RNN** est un réseau qui, à chaque pas de temps t, combine :

- $\mathbf{x}_t$: l'entrée **actuelle**
- $\mathbf{a}_{t-1}$: la **mémoire** du passé (état caché précédent)

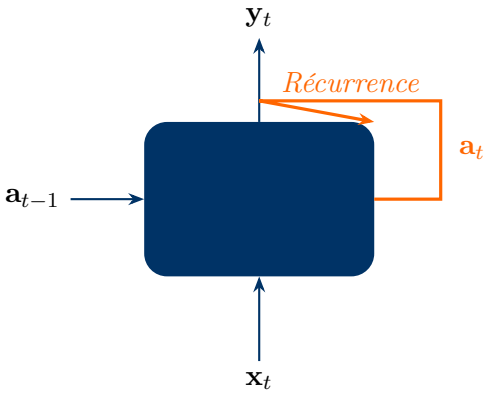
pour produire :

- $\mathbf{a}_t$: la mémoire **mise à jour**
- $\mathbf{y}_t$: la sortie **optionnelle** pour ce pas de temps

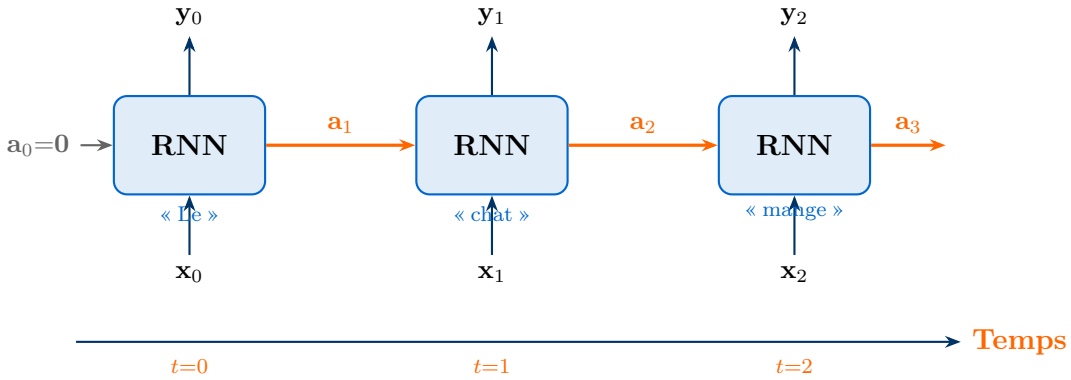
La **récurrence** est la boucle $\mathbf{a}_{t-1} \to \mathbf{a}_t$: c'est elle qui permet de **propager le contexte** d'un pas à l'autre.

3.2 La cellule RNN : forme repliée et dépliée

(a) Forme repliée — vue compacte avec la boucle de récurrence



(b) Forme dépliée — comment le réseau se déroule dans le temps



3.3 L'équation mathématique du RNN

L'équation fondamentale du RNN vanille

À chaque pas de temps t, la cellule RNN effectue un seul calcul :

$$\boxed{\mathbf{a}_t = \tanh\big(W_{aa}\,\mathbf{a}_{t-1} + W_{xa}\,\mathbf{x}_t + \mathbf{b}_a\big)}$$

Décomposons cette formule :

- $W_{xa}\,\mathbf{x}_t$: « que dit l'entrée actuelle ? » — projection de l'entrée
- $W_{aa}\,\mathbf{a}_{t-1}$: « que dit ma mémoire du passé ? » — projection de l'état précédent
- $\tanh(\cdot)$: on comprime le résultat dans $[-1, +1]$ pour stabiliser les valeurs
- $\mathbf{a}_t$: la **nouvelle mémoire** — un mélange de passé et de présent

Les poids appris :

- $W_{aa} \in \mathbb{R}^{H\times H}$: comment la mémoire passée influence la nouvelle mémoire
- $W_{xa} \in \mathbb{R}^{H\times D}$: comment l'entrée actuelle influence la mémoire

Astuce : Le secret du partage de poids

Les matrices W_{aa} et W_{xa} sont **les mêmes pour tous les pas de temps**. Cela signifie que le RNN apprend une règle générale de traitement, applicable à n'importe quelle position dans la séquence, quelle que soit sa longueur. C'est exactement comme une règle de grammaire : elle s'applique partout.

4 L'Entraînement : Rétropropagation dans le Temps (BPTT)

4.1 Principe général

Pour entraîner un RNN, on utilise la même idée que pour un MLP classique : calculer l'erreur, puis propager le gradient **en arrière** pour ajuster les poids. Mais il y a une difficulté supplémentaire : le réseau est **déployé dans le temps**.

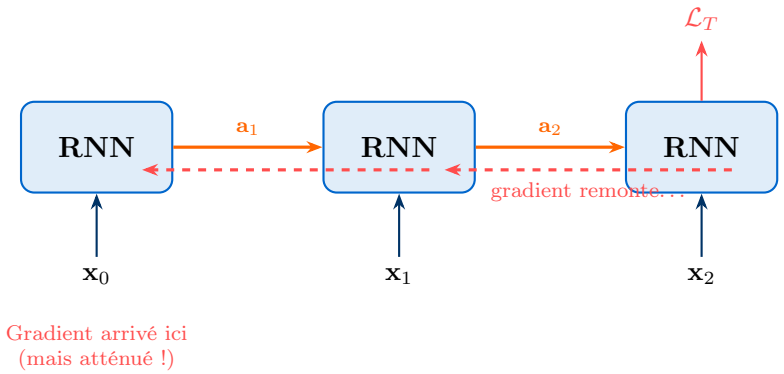
Backpropagation Through Time (BPTT)

La **rétropropagation dans le temps** consiste à :

1. **Dérouler** le RNN sur tous les pas de temps (comme dans la forme dépliée)
2. **Calculer** la perte totale $\mathcal{L} = \sum_t \mathcal{L}_t$
3. **Propager** le gradient de $\mathcal{L}$ en arrière à travers **tous les pas de temps**
4. **Sommer** les gradients par rapport à W (car W est partagé)

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial W}$$

4.2 Comment le gradient remonte le temps



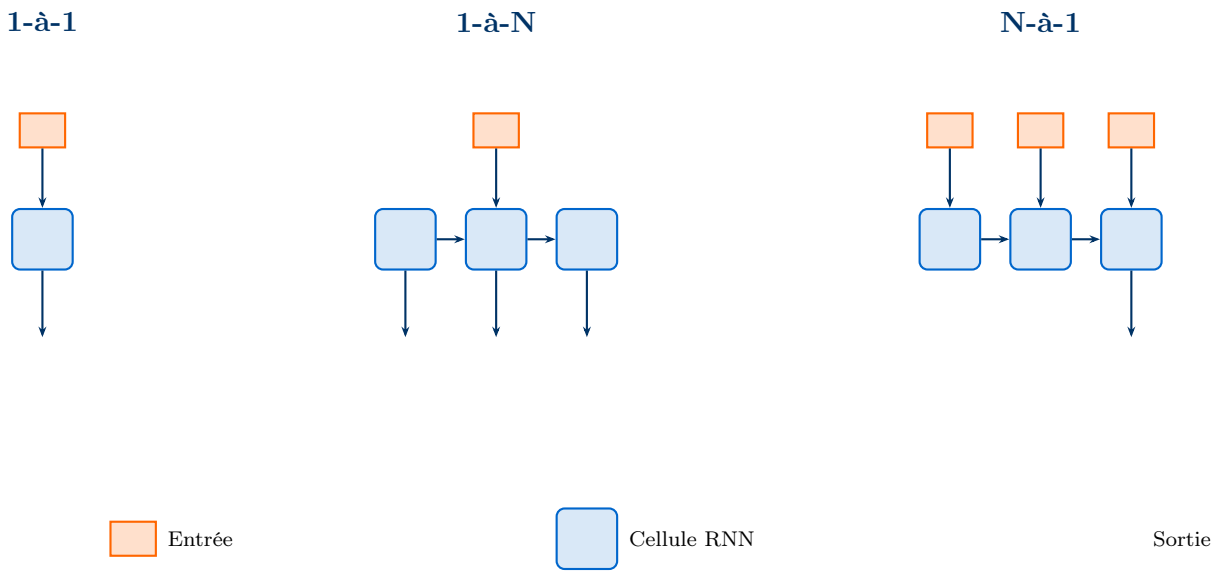
Attention : Le problème fondamental du BPTT

Lorsque le gradient remonte de $t{=}T$ jusqu'à $t{=}0$, il est **multiplié** par la matrice W_{aa} et par la dérivée de tanh **à chaque étape**. Sur une longue séquence, ce produit répété peut faire **disparaître** le gradient avant qu'il n'atteigne les premiers pas de temps.
C'est le célèbre **problème du gradient qui disparaît**, traité en détail à la Section 8.

5 Les Différentes Architectures RNN

5.1 Vue d'ensemble

Selon la tâche à accomplir, on connecte les entrées et les sorties d'un RNN de différentes manières. On distingue **5 architectures** :



5.2 Architecture 1-à-1 : la référence

Exemple : Un-à-Un

Description : une entrée, une sortie. Ce n'est pas vraiment un RNN — c'est simplement un réseau de neurones classique.
Exemple : classifier une image comme « chat » ou « chien ».
À retenir : ce mode est présenté uniquement comme point de comparaison.

5.3 Architecture 1-à-N : génération à partir d'un stimulus

Exemple : Un-à-Plusieurs

Description : une seule entrée produit une **séquence de sorties**. Le réseau reçoit un stimulus initial et génère une séquence entière. La sortie d'un pas de temps est réinjectée comme entrée au pas suivant.
Exemples concrets :

- **Génération de musique :** en entrée, un genre musical (« jazz »). En sortie, une séquence de notes musicales.
- **Légende d'image (Image Captioning) :** en entrée, les caractéristiques d'une image. En sortie, une description en mots : « Un chien court dans un parc. »

5.4 Architecture N-à-1 : classifier une séquence

Exemple : Plusieurs-à-Un

Description : une séquence en entrée, **une seule décision** en sortie. On utilise uniquement l'état caché **final** $\mathbf{a}_T$ (qui résume toute la séquence) pour produire la sortie.
Exemples concrets :

— **Analyse de sentiment :** entrée = phrase entière, sortie = positif / négatif.

« Ce film était absolument magnifique ! » $\longrightarrow$ **Positif (94%)**

— **Classification de spam :** entrée = email, sortie = spam / non-spam.

— **Détection d'intention :** entrée = question d'un utilisateur, sortie = catégorie (« réservation », « information »...)

5.5 Architecture N-à-N synchrone : annoter chaque élément

Exemple : Plusieurs-à-Plusieurs synchrone

Description : une séquence en entrée, une séquence de **même longueur** en sortie. Pour chaque entrée $\mathbf{x}_t$, on produit immédiatement une sortie $\mathbf{y}_t$.
Exemples concrets :

— **Étiquetage morpho-syntaxique (POS Tagging) :**

« Le »	« chat »	« mange »	« la »	« souris »
↓	↓	↓	↓	↓
DET	NOM	VERBE	DET	NOM

— **Reconnaissance d'entités nommées (NER) :**

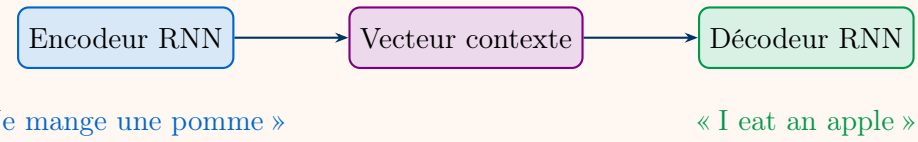
« Ali »	« habite »	« à »	« Alger »	« Algérie »
↓	↓	↓	↓	↓
PERSONNE	O	O	VILLE	PAYS

— **Annotation de vidéo :** pour chaque image, indiquer l'activité en cours (« marche », « court », « saute »).

5.6 Architecture N-à-N décalé (Seq2Seq) : transformer une séquence

Exemple : Plusieurs-à-Plusieurs décalé

Description : une séquence en entrée, une séquence de longueur **différente** en sortie. L'**encodeur** résume la séquence d'entrée en un vecteur de contexte, que le **décodeur** utilise pour générer la sortie.



Exemples concrets :

- **Traduction automatique :** « Bonjour, comment vas-tu ? » → « Hello, how are you ? »
- **Résumé automatique :** article de 500 mots → résumé de 50 mots
- **Chatbot :** question de l'utilisateur → réponse du bot

6 Modèles de Langage et Génération de Séquences

6.1 Qu'est-ce qu'un modèle de langage ?

Modèle de Langage

Un **modèle de langage** est un modèle qui apprend à **estimer la probabilité du prochain élément** d'une séquence, étant donné les éléments précédents :

$$P(\mathbf{x}_{t+1} \mid \mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_t)$$

En d'autres termes : « **Quel mot (ou caractère) est le plus probable après ce que j'ai déjà vu ?** »

Exemple : Complétion de texte au quotidien

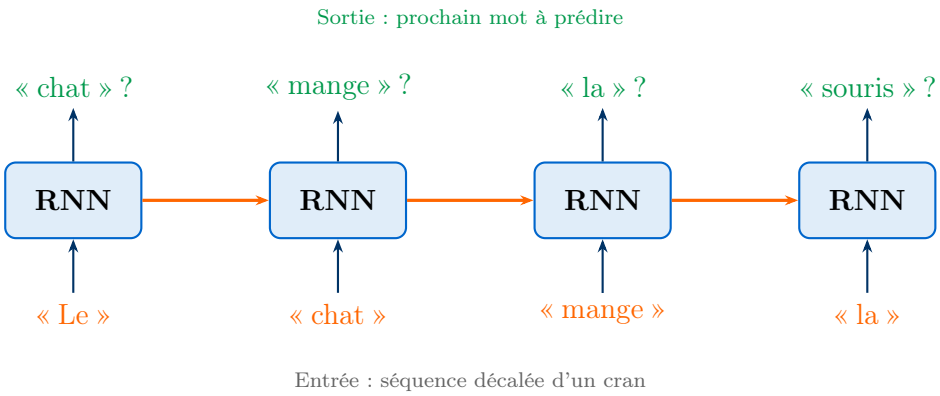
Le clavier de votre téléphone est un modèle de langage ! Lorsque vous tapez « Bonne », il vous suggère « journée », « nuit » ou « chance ». Il estime la probabilité des mots suivants selon votre historique d'écriture.

6.2 Niveau caractère ou niveau mot ?

Type	Unité traitée	Avantage / Inconvénient
Caractère	Chaque lettre : « B », « o », « n », « j »…	Vocabulaire minuscule ($\approx$ 100) ; séquences très longues
Mot	Chaque mot : « Bonjour », « monde »…	Plus naturel ; vocabulaire énorme ($\approx$ 50 000)

6.3 Comment un RNN devient un modèle de langage ?

L'entraînement est d'une simplicité remarquable : **à chaque pas de temps, prédire le prochain élément**.



Astuce : Entraînement auto-supervisé

La beauté d'un modèle de langage : **pas besoin d'étiquettes humaines !** Le texte lui-même est la supervision. Le texte « Le chat mange la souris » fournit automatiquement 4 paires (entrée, cible) :

Entrée	Cible
« Le »	« chat »
« Le chat »	« mange »
« Le chat mange »	« la »
« Le chat mange la »	« souris »

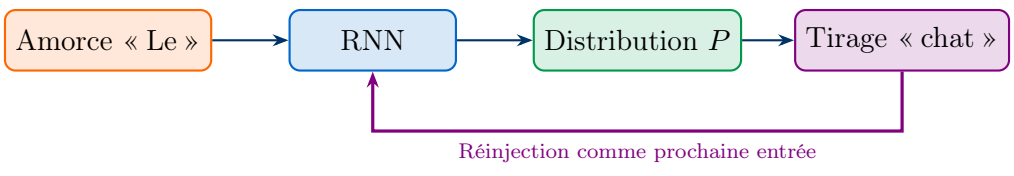
7 Sampling : Générer de Nouvelles Séquences

7.1 Du modèle appris à la génération

Une fois le modèle de langage entraîné, on peut l'utiliser pour **générer** de nouvelles séquences. On fournit une **amorce** (seed), et le modèle génère les éléments suivants un par un.

Processus de génération (sampling)

1. **Initialisation :** fournir une amorce (ex : « Le »)
2. **Prédiction :** le RNN calcule une distribution de probabilité sur tous les mots du vocabulaire
3. **Échantillonnage :** tirer aléatoirement un mot selon cette distribution
4. **Réinjection :** le mot tiré devient la prochaine entrée
5. **Répétition :** retourner à l'étape 2 jusqu'à la longueur souhaitée



7.2 La température : contrôler la créativité

Température de sampling

La **température** τ contrôle l'« audace » du modèle lors du tirage. On divise les scores (logits) par τ avant le softmax :

$$P(x_{t+1} = k) = \frac{\exp(z_k/\tau)}{\sum_j \exp(z_j/\tau)}$$

Température	Effet	Exemple de génération
$\tau \to 0$ (très basse)	Glouton : toujours le mot le plus probable. Texte très répétitif.	« Le chat mange. Le chat mange. Le chat... »
$\tau = 1$ (standard)	Distribution naturelle apprise. Équilibre cohérence / variété.	« Le chat mange la souris dans le jardin. »
$\tau > 1$ (haute)	Créatif mais incohérent. Mots rares plus souvent choisis.	« Le chat philosophique mange des étoiles bleues. »

Analogie du monde réel : L'étudiant et la température

- **Température basse** = étudiant très prudent, qui répond toujours ce qu'il sait avec certitude. Jamais de fautes, mais jamais de surprises.
- **Température haute** = étudiant créatif et audacieux, qui fait des associations inattendues. Parfois brillant, parfois incohérent.

8 Le Problème du Gradient qui Disparaît

8.1 Rappel : comment les gradients circulent dans un RNN

Pendant la BPTT, le gradient de la perte doit **remonter** de $t = T$ jusqu'à $t = 0$ pour mettre à jour les poids liés aux premiers pas de temps. À chaque étape de remontée, le gradient est multiplié par :

$$\frac{\partial \mathbf{a}_t}{\partial \mathbf{a}_{t-1}} = W_{aa}^\top \cdot \text{diag}\big(\tanh'(\mathbf{a}_t)\big)$$

Sur une séquence de T pas de temps, ce produit devient :

$$\frac{\partial \mathbf{a}_T}{\partial \mathbf{a}_0} = \prod_{t=1}^{T} W_{aa}^\top \cdot \text{diag}\big(\tanh'(\mathbf{a}_t)\big)$$

8.2 Pourquoi le gradient disparaît

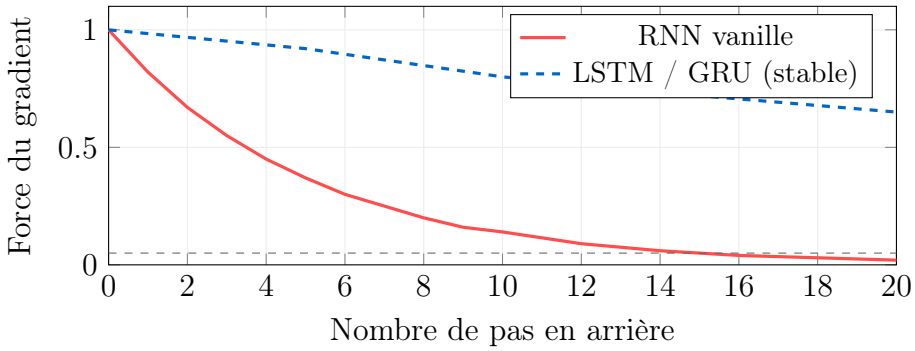
Gradient qui Disparaît (*Vanishing Gradient*)

Deux facteurs se combinent pour faire disparaître le gradient :

1. **La dérivée de** tanh est au maximum 1, et souvent bien plus petite. Pour les neurones saturés, $\tanh'(a) \approx 0$.
2. **La multiplication répétée** : si chaque facteur est < 1, le produit de T facteurs tend **exponentiellement vers zéro** :

$$\|\text{gradient}\| \leq \lambda^T \quad \text{avec} \quad \lambda < 1 \quad \Rightarrow \quad \text{gradient} \to 0 \text{ quand } T \to \infty$$

Le gradient disparaît exponentiellement dans le RNN vanille



8.3 Conséquence concrète : mémoire courte

Exemple : Dépendances longues impossibles

Supposons que le modèle doit apprendre la phrase :

« Les **nuages** dans le ciel sont sombres, il a plu hier, et maintenant le sol est encore humide à cause des _____. »

La réponse est « **nuages** », mais ce mot est à 15 pas de distance. Le gradient lié à « nuages » aura presque **disparu** avant d'atteindre le début de la séquence. Le RNN vanille sera incapable d'établir ce lien !

8.4 L'autre problème : le gradient qui explose

Attention : Gradient qui explose (*Exploding Gradient*)

Si les valeurs propres de W_{aa} sont > 1, le gradient **explose exponentiellement**. Le réseau devient instable (poids $\to \pm\infty$, valeurs `NaN` dans les pertes).

Solution simple : le gradient clipping

$$\text{Si } \|\nabla\| > \text{seuil}, \quad \nabla \leftarrow \frac{\text{seuil}}{\|\nabla\|} \cdot \nabla$$

On plafonne simplement la norme du gradient à une valeur maximale (typiquement 1.0).

Cette technique est indispensable en pratique pour tous les modèles récurrents.

8.5 Résumé : quand le RNN vanille atteint ses limites

Longueur de séquence	RNN vanille	Solution
Courte ($T < 10$)	Fonctionne bien	RNN suffit
Moyenne ($10 < T < 30$)	Difficultés	GRU ou LSTM recommandé
Longue ($T > 30$)	Quasi-impossible	GRU ou LSTM obligatoire
Très longue ($T > 100$)	Impossible	LSTM ou Transformer

9 GRU — Gated Recurrent Unit

9.1 Motivation : résoudre le problème avec des portes

En 2014, Cho et al. ont proposé le **GRU** pour résoudre le problème du gradient qui disparaît. L'idée centrale : introduire des **portes** — des mécanismes qui décident **quoi garder** et **quoi oublier** dans la mémoire.

Analogie du monde réel : La porte d'un immeuble

Imaginez la mémoire d'un RNN comme un couloir. Dans le RNN vanille, toutes les informations passent et s'atténuent progressivement. Le GRU ajoute des **portes intelligentes** : certaines informations passent librement (« garder »), d'autres sont bloquées (« oublier »). La mémoire importante peut ainsi voyager sur de longues distances sans s'affaiblir.

9.2 Les deux portes du GRU

Architecture GRU — Les trois équations

Le GRU utilise **deux portes** et un **état candidat** :

1. Update gate Γ_u — « Combien garder du passé ? »

$$\Gamma_u = \sigma\big(W_u \cdot [\mathbf{a}_{t-1}, \mathbf{x}_t]\big) \qquad \in [0,1]^H$$

2. Reset gate Γ_r — « Quel passé utiliser pour calculer la proposition ? »

$$\Gamma_r = \sigma\big(W_r \cdot [\mathbf{a}_{t-1}, \mathbf{x}_t]\big) \qquad \in [0,1]^H$$

3. Mise à jour finale (interpolation intelligente) :

$$\tilde{\mathbf{C}}_t = \tanh\big(W \cdot [\Gamma_r \odot \mathbf{a}_{t-1}, \mathbf{x}_t]\big)$$

$$\boxed{\mathbf{a}_t = (1 - \Gamma_u) \odot \mathbf{a}_{t-1} + \Gamma_u \odot \tilde{\mathbf{C}}_t}$$

9.3 Interprétation intuitive

L'équation finale du GRU est une **interpolation linéaire** entre le passé et la nouvelle proposition :

Cas	Valeur de Γ_u	Comportement
Mémorisation	$\Gamma_u \approx 0$	$\mathbf{a}_t \approx \mathbf{a}_{t-1}$: on **conserve** la mémoire intacte
Mise à jour	$\Gamma_u \approx 1$	$\mathbf{a}_t \approx \tilde{\mathbf{C}}_t$: on **remplace** par la nouvelle information

Astuce : La magie du GRU

La porte apprend **automatiquement**, pendant l'entraînement, quand mémoriser et quand mettre à jour. Pour les mots importants du contexte, Γ_u sera proche de 0 (mémorisation). Pour les informations nouvelles et pertinentes, Γ_u sera proche de 1 (mise à jour).

9.4 Pourquoi le GRU résout le gradient qui disparaît

Grâce à la connexion directe entre $\mathbf{a}_{t-1}$ et $\mathbf{a}_t$ via $(1 - \Gamma_u)$, le gradient peut **court-circuiter** plusieurs pas de temps et remonter directement jusqu'aux états anciens **sans multiplication répétée par** W_{aa}. L'information et le gradient peuvent ainsi voyager sur de longues distances.

10 LSTM — Long Short-Term Memory

10.1 Motivation : une mémoire à deux niveaux

En 1997, Hochreiter et Schmidhuber ont proposé le **LSTM** — bien avant le GRU. L'idée : séparer clairement la **mémoire à long terme** (l'état de cellule $\mathbf{C}_t$) de la **mémoire à court terme** (l'état caché $\mathbf{a}_t$).

Analogie du monde réel : Un employé avec un carnet de notes

Imaginez un secrétaire pendant une longue réunion :

- Son **carnet** ($\mathbf{C}_t$) : les informations importantes notées, qui persistent longtemps sans s'effacer.
- Sa **mémoire de travail** ($\mathbf{a}_t$) : ce qu'il vient d'entendre et utilise immédiatement pour répondre.
- À tout moment, il peut **effacer** des notes périmées, **ajouter** de nouvelles informations importantes, et **consulter** ses notes pour prendre une décision.

Les 3 portes du LSTM imitent exactement ces mécanismes.

10.2 Les trois portes du LSTM

Architecture LSTM — Les quatre équations fondamentales

Le LSTM utilise **trois portes** contrôlant un **état de cellule** $\mathbf{C}_t$:

1. Forget gate Γ_f — « Qu'efface-t-on du carnet ? »

$$\Gamma_f = \sigma\big(W_f \cdot [\mathbf{a}_{t-1},\, \mathbf{x}_t] + \mathbf{b}_f\big) \qquad \in [0,1]^H$$

2. Input gate Γ_i — « Quoi écrire dans le carnet ? »

$$\Gamma_i = \sigma\big(W_i \cdot [\mathbf{a}_{t-1},\, \mathbf{x}_t] + \mathbf{b}_i\big) \qquad \tilde{\mathbf{C}}_t = \tanh\big(W_C \cdot [\mathbf{a}_{t-1},\, \mathbf{x}_t] + \mathbf{b}_C\big)$$

3. Mise à jour du carnet (l'autoroute de l'information) :

$$\boxed{\mathbf{C}_t = \underbrace{\Gamma_f \odot \mathbf{C}_{t-1}}_{\text{ce qu'on garde}} + \underbrace{\Gamma_i \odot \tilde{\mathbf{C}}_t}_{\text{ce qu'on écrit}}}$$

4. Output gate Γ_o — « Que communique-t-on à l'extérieur ? »

$$\Gamma_o = \sigma\big(W_o \cdot [\mathbf{a}_{t-1},\, \mathbf{x}_t] + \mathbf{b}_o\big) \qquad \mathbf{a}_t = \Gamma_o \odot \tanh(\mathbf{C}_t)$$

Où $\odot$ = produit élément par élément, $[\cdot,\cdot]$ = concaténation, σ = sigmoïde.

10.3 Pourquoi le LSTM résout le gradient qui disparaît

Astuce : L'astuce de la mise à jour additive

La clé est l'équation de l'état de cellule :

$$\mathbf{C}_t = \Gamma_f \odot \mathbf{C}_{t-1} + \Gamma_i \odot \tilde{\mathbf{C}}_t$$

Le gradient de la perte par rapport à $\mathbf{C}_{t-1}$ est simplement Γ_f, qui peut rester **proche de 1**.
Contrairement au RNN vanille où le gradient est **multiplié** à chaque pas par $W_{aa}^{\top} \cdot \tanh'(\cdot)$, ici le gradient **s'additionne** via l'état de cellule. C'est cette différence fondamentale qui permet au LSTM de maintenir des dépendances à très longue distance.

10.4 Comparaison finale : RNN, GRU, LSTM

Critère	RNN vanille	GRU	LSTM
Nombre de portes	0	2	3
États internes	$\mathbf{a}_t$ seulement	$\mathbf{a}_t$ seulement	$\mathbf{a}_t$ et $\mathbf{C}_t$
Complexité	Simple	Modérée	Plus complexe
Mémoire longue	Très limitée	Bonne	Très bonne
Vitesse	Rapide	Rapide	Plus lent
Conseil	Éviter pour NLP	Premier essai	Séquences longues

11 Récapitulatif et Points Clés

Résumé : Le parcours complet en 9 points essentiels

1. **Modèles de séquences :** dans les données séquentielles, l'ordre est fondamental. Le sens émerge du contexte.
2. **Limites des MLP :** taille d'entrée fixe, aucune mémoire, insensibles à l'ordre — incapables de traiter les séquences.
3. **RNN :** introduit l'état caché $\mathbf{a}_t$ — une mémoire qui se met à jour à chaque pas. $\mathbf{a}_t = \tanh(W_{aa}\mathbf{a}_{t-1} + W_{xa}\mathbf{x}_t + \mathbf{b})$.
4. **BPTT :** entraînement par déroulement dans le temps. Les poids partagés reçoivent la somme des gradients de tous les pas.
5. **5 architectures :** 1-à-1, 1-à-N, N-à-1, N-à-N synchrone, N-à-N décalé (Seq2Seq) — chacune adaptée à un type de tâche.
6. **Modèle de langage :** prédire le prochain élément. Entraînement auto-supervisé : le texte lui-même est la supervision.
7. **Sampling :** générer des séquences en réinjectant les sorties. La température τ contrôle la créativité.

8. **Gradient qui disparaît :** le produit répété de matrices atténue le gradient exponentiellement. Le RNN « oublie » le passé lointain ($T > 20$).
9. **GRU et LSTM :** des portes créent des connexions directes permettant aux gradients de voyager loin sans s'atténuer.
   — **GRU** : 2 portes, plus simple, bonne performance générale.
   — **LSTM** : 3 portes + état de cellule, meilleur sur séquences très longues.



AI DZ ACADEMY

Apprenez l'Intelligence Artificielle, simplement et efficacement

Contact : contact@aidzacademy.org

WhatsApp : +213 557 90 89 57

Instagram : @ai_dz_academy

Site Web : www.aidzacademy.org

Alger, Algérie | © 2026 AI DZ Academy